

**TUGAS PRAKTIKUM PERTEMUAN 2**  
**PENGANTAR BAHASA PEMROGRAMAN DART**  
**PEMROGRAMAN MOBILE**



**NAMA : Aamira Faheema Ghania**  
**NIM : 244107060081**  
**KELAS : SIB 2D**  
**ABSEN : 01**

**JURUSAN TEKNOLOGI INFORMASI**  
**PROGRAM STUDI SISTEM INFORMASI BISNIS**  
**POLITEKNIK NEGERI MALANG**

**2026**

## Tugas Praktikum

1. Modifikasilah kode pada baris 3 di VS Code atau Editor Code favorit Anda berikut ini agar mendapatkan keluaran (*output*) sesuai yang diminta!

```
1 void main() {  
2     for (int i = 0; i < 10; i++) {  
3         print('hello ${i + 2}');  
4     }  
5 }
```

Output yang diminta (Gantilah **Fulan** dengan nama Anda):

```
Console  
  
Nama saya adalah Fulan, sekarang berumur 18  
Nama saya adalah Fulan, sekarang berumur 17  
Nama saya adalah Fulan, sekarang berumur 16  
Nama saya adalah Fulan, sekarang berumur 15  
Nama saya adalah Fulan, sekarang berumur 14  
Nama saya adalah Fulan, sekarang berumur 13  
Nama saya adalah Fulan, sekarang berumur 12  
Nama saya adalah Fulan, sekarang berumur 11  
Nama saya adalah Fulan, sekarang berumur 10  
Nama saya adalah Fulan, sekarang berumur 9
```

Jawab:

Kode Program:

```
1 void main() {  
2     for (int i = 18; i > 8; i--) {  
3         print('Nama saya adalah Aamira, sekarang berumur ${i}');  
4     }  
5 }
```

Output:

```
Nama saya adalah Aamira, sekarang berumur 18  
Nama saya adalah Aamira, sekarang berumur 17  
Nama saya adalah Aamira, sekarang berumur 16  
Nama saya adalah Aamira, sekarang berumur 15  
Nama saya adalah Aamira, sekarang berumur 14  
Nama saya adalah Aamira, sekarang berumur 13  
Nama saya adalah Aamira, sekarang berumur 12  
Nama saya adalah Aamira, sekarang berumur 11  
Nama saya adalah Aamira, sekarang berumur 10  
Nama saya adalah Aamira, sekarang berumur 9
```

Kode tersebut merupakan program Dart yang dijalankan melalui fungsi `main()` sebagai titik awal eksekusi. Di dalamnya terdapat perulangan `for` yang dimulai dari nilai `i = 18` dan akan terus berjalan selama nilai `i` lebih dari 8, dengan setiap iterasi nilai `i` dikurangi satu (`i--`). Pada setiap pengulangan, program mencetak kalimat “Nama saya adalah Aamira, sekarang berumur `${i}`” ke layar, di mana `${i}` menggunakan string interpolation untuk menampilkan nilai variabel `i`. Akibatnya, program akan menampilkan umur dari 18 hingga 9 secara menurun.

2. Mengapa sangat penting untuk memahami bahasa pemrograman Dart sebelum kita menggunakan framework Flutter ? Jelaskan!

Jawab:

Memahami bahasa pemrograman Dart sangat penting sebelum menggunakan framework Flutter karena Flutter sepenuhnya dibangun menggunakan Dart sebagai Bahasa utamanya. Jika kita belum memahami dasar-dasar Dart serta konsep OOP, maka kita akan kesulitan membaca, menulis, dan memperbaiki kode Flutter. Selain itu, pemahaman Dart membantu kita memahami bagaimana Flutter bekerja di balik layar.

3. Rangkumlah materi dari codelab ini menjadi poin-poin penting yang dapat Anda gunakan untuk membantu proses pengembangan aplikasi mobile menggunakan framework Flutter.

Jawab:

1. Dart merupakan bahasa utama yang digunakan oleh framework Flutter
2. Beberapa fitur penting Dart yang mendukung pengembangan Flutter:
  - Productive tooling → Dukungan IDE, analisis kode, dan ekosistem paket yang kuat.
  - Garbage collection → Manajemen memori otomatis.
  - Type annotations (opsional) → Mendukung keamanan tipe data.
  - Statically typed & type-safe → Mendeteksi bug sejak kompilasi.
  - Type inference → Tidak selalu perlu menulis tipe secara eksplisit.
  - Portability → Bisa dikompilasi ke:
    - JavaScript (untuk web)
    - Native ARM & x86 (untuk mobile/desktop)

Ini membuat Flutter bisa berjalan lintas platform dengan performa tinggi.

3. Dart dapat dijalankan dengan dua pendekatan utama:

- JIT (Just-In-Time)
  - Digunakan saat development
  - Mendukung debugging dan hot reload.
  - Cocok untuk pengembangan cepat.
- AOT (Ahead-Of-Time)
  - Digunakan saat production.
  - Dikompilasi langsung ke native machine code.
  - Performa lebih tinggi, tetapi tanpa hot reload.

Flutter memanfaatkan JIT untuk hot reload dan AOT untuk performa aplikasi saat dirilis.

4. Hot Reload (Fitur penting Flutter)

- Perubahan kode langsung terlihat tanpa restart aplikasi
- Didukung oleh Dart JIT compiler

5. Dart sepenuhnya berbasis OOP. Semua data di Dart adalah objek (tidak ada tipe primitif seperti di Java)
6. Operator penting dalam Dart:

- Arithmetic  
+, -, \*, /, ~/ , %, +=, -=, dll.  
/ selalu menghasilkan double  
~/ untuk pembagian bilangan bulat
- Logical  
!, &&, ||
- Increment & Decremen  
++, --
- Equality & Relational  
==, !=, >, <, >=, <=  
Berbeda dari Java:  
== membandingkan isi, bukan referensi memori.  
Tidak ada === seperti di JavaScript karena sudah type-safe.

#### 7. Function vs Method

- Function → berdiri sendiri (contoh: main())
- Method → Terikat pada class dan memiliki akses ke this

#### 4. Buatlah penjelasan dan contoh eksekusi kode tentang perbedaan *Null Safety* dan *Late variabel* !

Jawab:

1. Null Safety → fitur di Dart yang mencegah variabel memiliki nilai null kecuali memang diizinkan. Tujuannya untuk mengurangi error seperti Null Pointer Exception saat program berjalan.

Contoh tanpa Nullable (Tidak boleh null)

```
Run | Debug | Qodo: Test this function
1 void main() {
2   String nama = 'Aamira';
3   print(nama);
4 }
```

Output:

```
Aamira
```

Dalam bahasa pemrograman Dart, kode tanpa nullable berarti variabel tidak diperbolehkan memiliki nilai null. Ketika menulis `String nama = "Aamira";`, variabel `nama` wajib langsung diinisialisasi dan selalu memiliki nilai bertipe `String`. Jika variabel dideklarasikan tanpa tanda `?` tetapi tidak diberi nilai, maka akan terjadi error saat proses kompilasi karena Dart dengan `Null Safety` mengharuskan variabel `non-nullable` memiliki nilai yang jelas.

### Contoh Nullable (Boleh null)

```
1 void main() {  
2   String? nama;  
3   print(nama);  
4 }
```

Output:

```
null
```

Kode nullable ditandai dengan tanda tanya (?) seperti pada `String? nama;`, yang berarti variabel tersebut diperbolehkan bernilai null. Dalam hal ini, Dart tidak akan menghasilkan error meskipun variabel belum diberi nilai awal, karena null dianggap sebagai nilai yang valid.

2. Late Variabel → late digunakan ketika variabel tidak langsung diinisialisasi, tetapi pasti akan diberi nilai sebelum digunakan.

### Contoh tanpa late

```
1 void main() {  
2   String nama;  
3   print(nama);  
4 }
```

Output:

```
PS C:\portfolio\PemrogramanMobile> dart "c:\portfolio\PemrogramanMobile\Pertemuan2\soal4_tanpa-late.dart"  
Pertemuan2\soal4_tanpa-late.dart:3:9: Error: Non-nullable variable 'nama' must be assigned before it can be used.  
  print(nama);  
      ^^^^^
```

Pada kode tersebut variabel dideklarasikan tanpa langsung diberi nilai, misalnya `String nama;` lalu langsung dipanggil dengan `print(nama);`. Karena Dart menerapkan Null Safety, variabel bertipe non-nullable seperti `String` wajib diinisialisasi sebelum digunakan. Jika tidak, akan terjadi error saat compile time karena Dart menganggap variabel tersebut berpotensi belum memiliki nilai.

### Contoh dengan late

```
1 void main() {  
2   late String nama;  
3   nama = 'Aamira';  
4   print(nama);  
5 }
```

Output:

```
Aamira
```

Pada kode program tersebut, variabel dideklarasikan menggunakan kata kunci `late`, seperti `late String nama;`, yang berarti inisialisasi nilainya ditunda. Meskipun tidak langsung diberi nilai saat deklarasi, Dart memperbolehkannya karena kita menjamin bahwa variabel tersebut akan diisi sebelum digunakan. Ketika nilai

kemudian diberikan, misalnya `nama = "Aamira";`, dan setelah itu dipanggil dengan `print(nama);`, program akan berjalan normal. Namun, jika variabel `late` digunakan sebelum diberi nilai, maka akan muncul error saat runtime karena variabel belum diinisialisasi.

Contoh error pada `late`

```
1 void main() {  
2   late String nama;  
3   print(nama);  
4 }
```

Output:

```
Pertemuan2/soal4_error-late.dart:3:9: Error: Late variable 'nama' without initializer is definitely unassigned.  
  print(nama);  
      ^^^^^
```

Pada kode tersebut, variabel dideklarasikan menggunakan `late String nama;` tetapi langsung dipanggil dengan `print(nama);` tanpa pernah diberi nilai terlebih dahulu. Karena `late` hanya menunda inisialisasi (bukan mengizinkan `null`), Dart menganggap variabel tersebut pasti akan diisi sebelum digunakan. Namun saat program dijalankan dan variabel belum memiliki nilai, akan muncul `LateInitializationError` pada runtime. Artinya, kesalahan tidak terdeteksi saat compile time, tetapi baru muncul ketika program dieksekusi.